Name:	**Jay Chandrasen Talekar**
Roll No:	**50**
Class/Sem:	TE/VI
Experiment No.:	06
Title:	To learn dockerfile instructions build an image for a sample web application using dockerfile
Date of Performance:	
Date of Submission:	
Marks: **Sign of Faculty:**	

Aim: To learn dockerfile instructions build an image for a sample web application using dockerfile

Theory:

Dockerfile:

A Dockerfile is a text configuration file written using a special syntax. It describes step-by-step instructions of all the commands you need to run to assemble a Docker Image. The docker build command processes this file generating a Docker Image in your Local Image Cache, which you can then start-up using the docker run command, or push to a permanent Image Repository. Creating a Dockerfile is as easy as creating a new file named "Dockerfile" with your text editor of choice and defining some instructions. The name of the file does not really matter.

Docker Image:

A Docker image is a lightweight, standalone, executable package that contains everything needed to run an application or service, including the application code, libraries, dependencies, and operating system. It is a template that can be used to create Docker containers, which are instances of an image that can be run in isolation.

Here are some common Dockerfile instructions:

1. FROM - Specifies the base image to use for the build.
2. RUN - Executes a command inside the container during the build process. This is often used to install software or dependencies.
3. COPY - Copies files from the host system into the container.
4. ADD - Similar to COPY, but can also download files from a URL and extract compressed files.
5. WORKDIR - Sets the working directory inside the container for subsequent commands. 6. ENV - Sets environment variables inside the container.
7. EXPOSE - Informs Docker that the container listens on the specified network ports at runtime.
8. CMD - Specifies the command to run when the container is started.
9. ENTRYPOINT - Specifies the command to run when the container is started and allows arguments to be passed to the command.

Steps:

1. Create a new directory for your Dockerfile and application files.
2. Create a new file named Dockerfile in this directory.
3.Open the Dockerfile and write the instructions to build your Docker image.

Here's an example

```
# The line below states we will base our new image on the Latest Official Ubun
FROM ubuntu:latest

#
# Identify the maintainer of an image
LABEL maintainer="myname@somecompany.com"

#
# Update the image to the latest packages
RUN apt-get update && apt-get upgrade -y

#
# Install NGINX to test.
RUN apt-get install nginx -y

#
# Expose port 80
EXPOSE 80

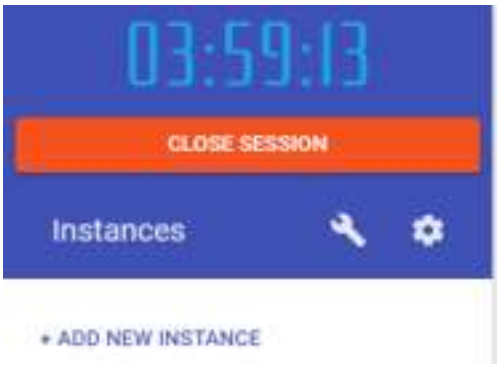
#
# Last is the actual command to start up NGINX within our Container
CMD ["nginx", "-g", "daemon off;"]
```

4. Save and close the docker file.

Now Building and Testing Dockerfiles

1. First of all, head over to http://play-with-docker.com and start a new session. You need to create an account first.

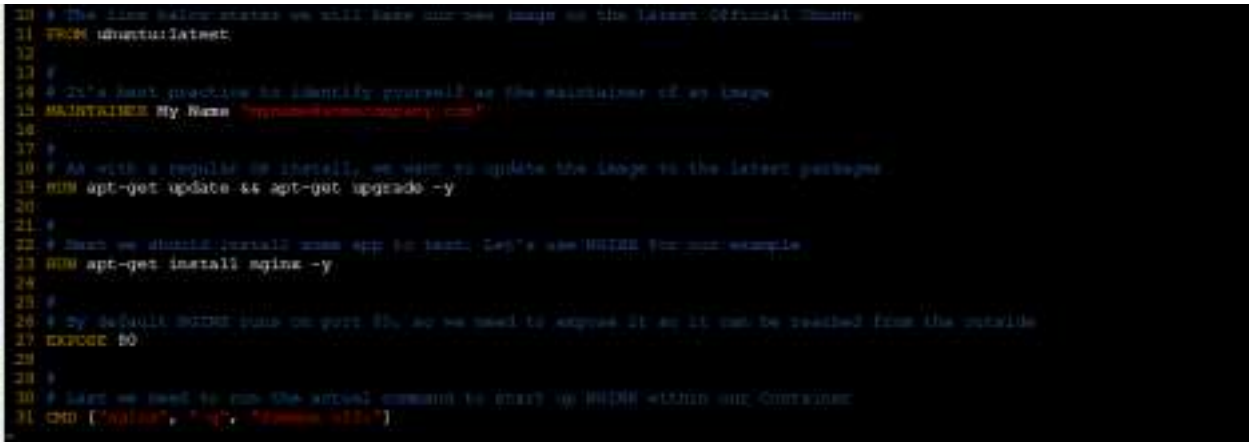
2. Once your session is active click on "Add New Instance":



3. A new instance will start with a Docker Engine ready to accept commands



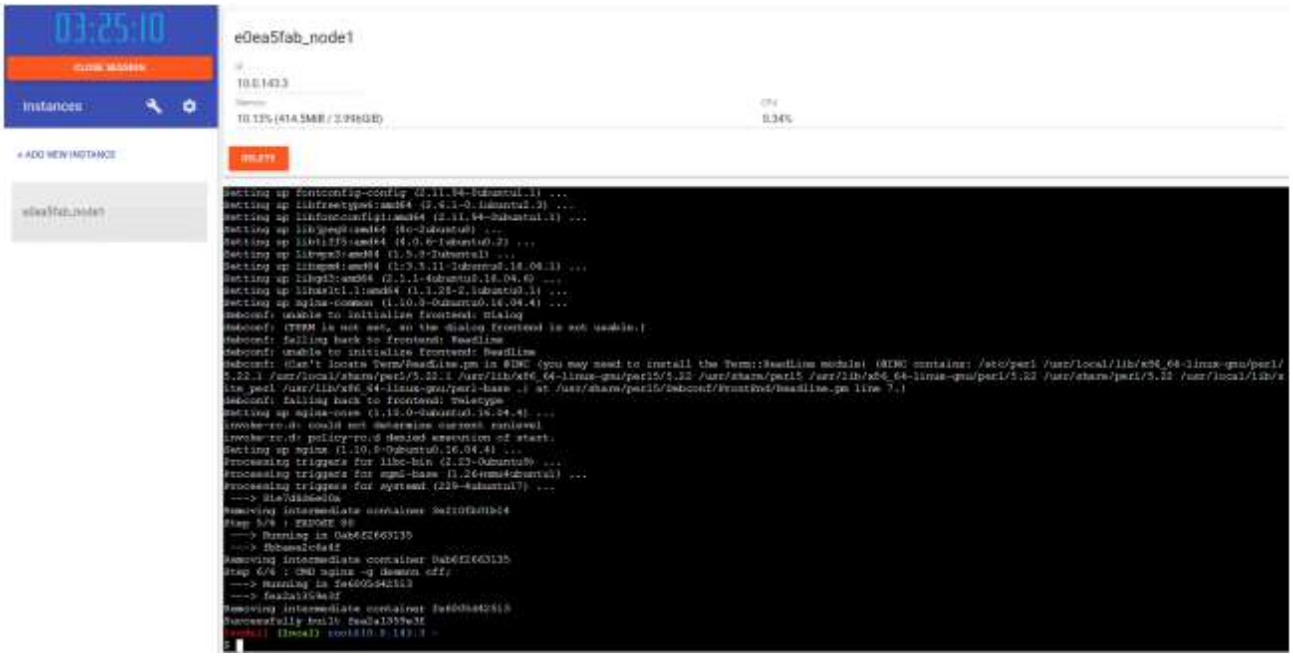
4. Next create/edit the Dockerfile. Run "vi Dockerfile", press "i" to switch to "Insert Mode", copy/paste the contents of our Dockerfile, press "Esc" to exit "Insert Mode", and save+exit by typing ":x"



5. Build the new image using the command docker build <path>. Path refers to the directory containing the Dockerfile.



6. At the end of the process you should see the message "Successfully built <image ID>"



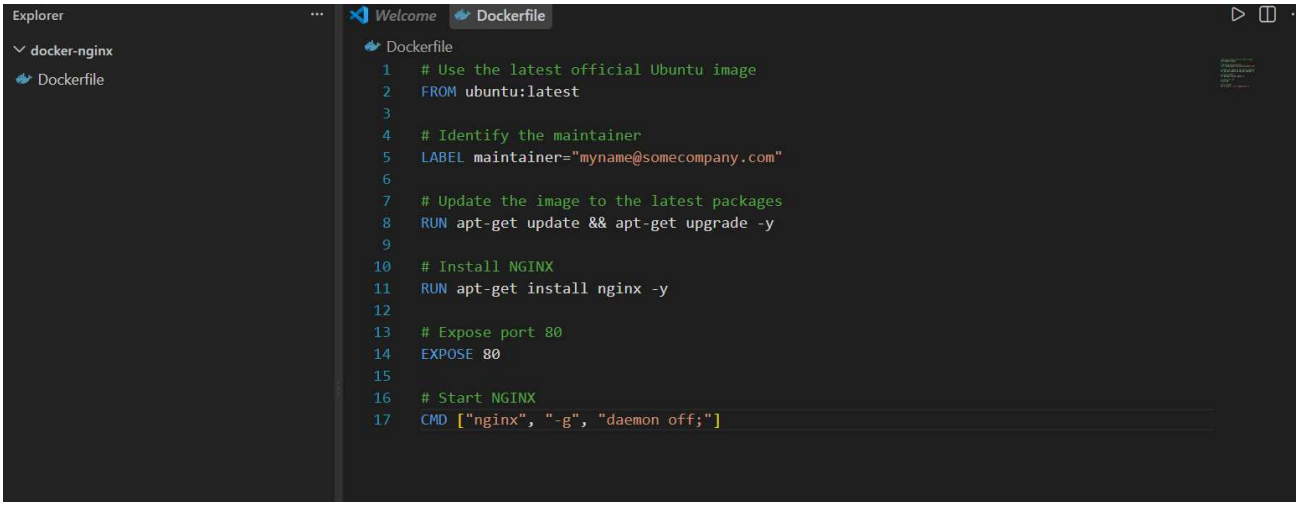
7. Start the new image and test connectivity to NGINX. Run the command
docker run -p 80:80 <image ID>. The option -p 80:80 exposes the Container port 80 as the Host port 80 to the world

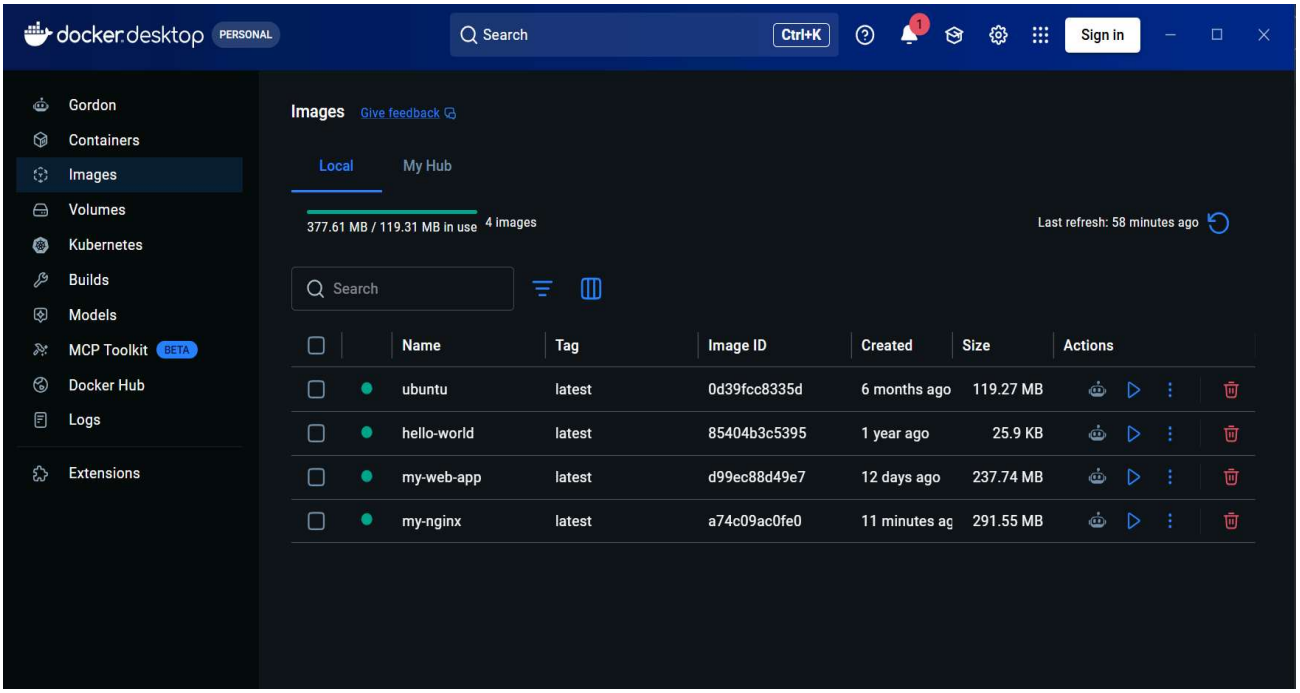


8. As a result a port 80 link should have become active next to the IP. Click on it to access your NGINX service.

9. That's it !!! We have created a docker image and run it in our local machine.

Output:


Explorer
docker-nginx
Dockerfile
Welcome
Dockerfile
Dockerfile
1 # Use the latest official Ubuntu image
2 FROM ubuntu:latest
3
4 # Identify the maintainer
5 LABEL maintainer="myname@somecompany.com"
6
7 # Update the image to the latest packages
8 RUN apt-get update && apt-get upgrade -y
9
10 # Install NGINX
11 RUN apt-get install nginx -y
12
13 # Expose port 80
14 EXPOSE 80
15
16 # Start NGINX
17 CMD ["nginx", "-g", "daemon off;"]


docker desktop
PERSONAL
Search
Ctrl+K
Sign in
Gordon
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Logs
Extensions
Images
Give feedback
Local
My Hub
377.61 MB / 119.31 MB in use
4 images
Last refresh: 58 minutes ago
Search
Name
Tag
Image ID
Created
Size
Actions
ubuntu latest 0d39fcc8335d 6 months ago 119.27 MB
hello-world latest 85404b3c5395 1 year ago 25.9 KB
my-web-app latest d99ec88d49e7 12 days ago 237.74 MB
my-nginx latest a74c09ac0fe0 11 minutes ag 291.55 MB

```
C:\Users\student\docker-nginx>docker build -t my-nginx .
[+] Building 549.2s (7/7) FINISHED                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.0s
 => => transferring dockerfile: 389B                                                0.0s
 => [internal] load metadata for docker.io/library/ubuntu:latest                    1.1s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [1/3] FROM docker.io/library/ubuntu:latest@sha256:0d39fcc8335d6d74d5502f6df2d30119ff4790ebb  0.0s
 => => resolve docker.io/library/ubuntu:latest@sha256:0d39fcc8335d6d74d5502f6df2d30119ff4790ebb  0.0s
 => [2/3] RUN apt-get update && apt-get upgrade -y                                493.5s
 => [3/3] RUN apt-get install nginx -y                                             49.1s
 => exporting to image                                                              5.2s
 => => exporting layers                                                             4.0s
 => => exporting manifest sha256:91a611d326ff14cebd31536477ba4aca4808156360630f9067df6cec10e1f4  0.0s
 => => exporting config sha256:de876a58a8fa741170ac2dca94462feb8ad4deb10e1b11507438d5fbb82fa9d4  0.0s
 => => exporting attestation manifest sha256:cbb751be8b4388aeb65a86010d78e89076ecba5fc62e35f625  0.0s
 => => exporting manifest list sha256:a74c09ac0fe09474eb881faf764dd88caed07f4bb01a5fe833e55c1fd  0.0s
 => => naming to docker.io/library/my-nginx:latest                                  0.0s
 => => unpacking to docker.io/library/my-nginx:latest                               1.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/kb52q3927bijr6lk0nrl0
ue61

C:\Users\student\docker-nginx>docker images
                                                                  i Info →   U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   85404b3c5395       25.9kB         9.52kB    U
my-nginx:latest      a74c09ac0fe0        292MB         88.6MB
my-web-app:latest    d99ec88d49e7        238MB         63.1MB    U
ubuntu:latest        0d39fcc8335d        119MB         31.7MB    U
```

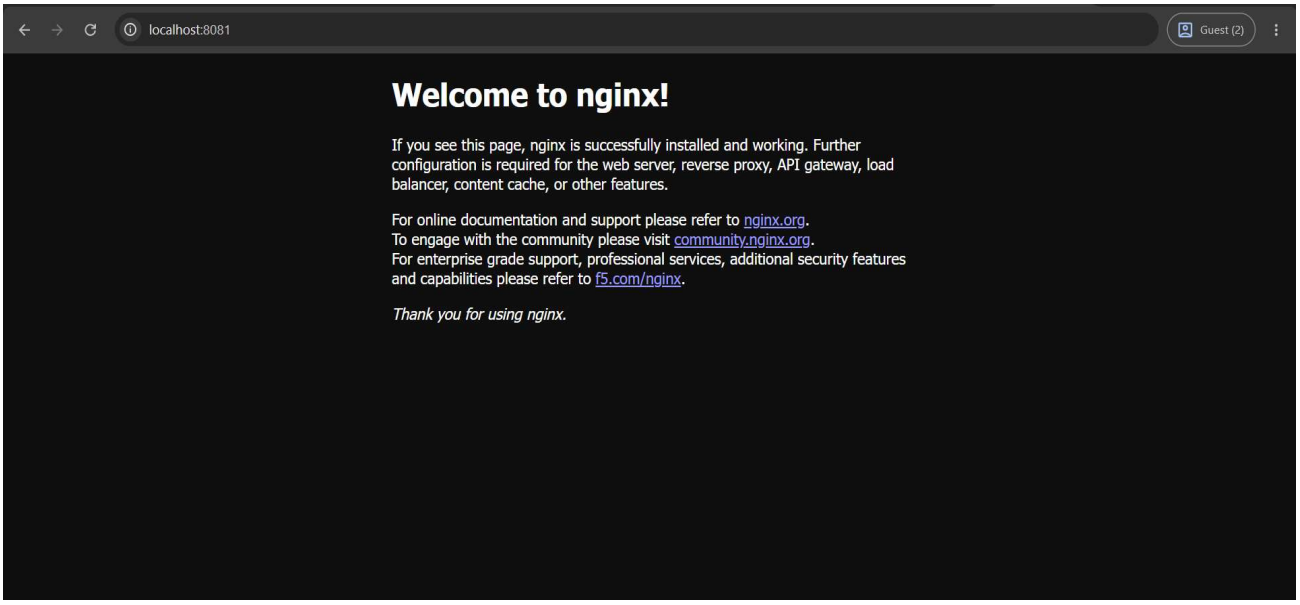
```
C:\Users\student\docker-nginx>docker rm nginx-container
nginx-container

C:\Users\student\docker-nginx>docker run -d -p 8081:80 --name nginx-container my-nginx
1d6f083cbb02ec1733505074a14ede205ebf6c2e46a7652b56cf40a0284845ba

What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
docker: Error response from daemon: failed to set up container networking: driver failed programming e
xternal connectivity on endpoint nginx-container (227f4291f176b2c53209e25c688d19d692fefb7edf227592d8b7
15d819b42634): Bind for 0.0.0.0:8081 failed: port is already allocated

Run 'docker run --help' for more information

C:\Users\student\docker-nginx>docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS          PORTS
                          NAMES
d68642cda623   my-web-app   "/docker-entrypoint.…"   36 minutes ago   Up 36 minutes   0.0.0.0:8081->80
/tcp, [::]:8081->80/tcp   my-container
```



Conclusion:

Q1. What is Dockerfile?

A **Dockerfile** is a text file that contains a set of instructions used to **create a Docker image**. It specifies the base image, software to install, ports to expose, and the command to run when the container starts.

Q2. What is DockerImage?

A **Docker Image** is a **read-only package/template** containing everything required to run an application, such as the application code, libraries, dependencies, and configuration. A Docker container is created from a Docker image.